Universidad de Buenos Aires

Facultad de Ingeniería

Redes Neuronales (8654)

Trabajo Practico N°1

2°C 2024

Alumno: Milidoni, Cristian Julian

Padrón: 107647

Septiembre 2024

Contents

1 Introducción **2**
1.1 Red de Hopfield: Entrenamiento y Evaluación . . . 2
1.2 Regla de Hebb para el Entrenamiento . . . 2
1.3 Evaluación de la Red . . . 2
1.3.1 Evaluación Sincrónica . . . 2
1.3.2 Evaluación Asincrónica . . . 3
1.4 Implementación de la Red de Hopfield . . . 3

2 Red Neuronal de Hopfield para set de imágenes 50x50 **4**
2.1 Entrenamiento . . . 4
2.2 Evaluación de la Red con imágenes recortadas o con ruido . . . 5
2.3 Estados Espurios . . . 6

3 Red Neuronal de Hopfield para set de imagenes 45x60 **8**
3.1 Entrenamiento . . . 8
3.2 Evaluación de la Red con imágenes recortadas o con ruido . . . 9
3.3 Estados Espurios . . . 10

4 Red Neuronal de Hopfield para set de imágenes de varios tamaños **12**

5 Capacidad de la Red Hopfield **14**

1 Introducción

El presente trabajo tiene como objetivo implementar, entrenar y evaluar una red neuronal de Hopfield utilizando diferentes conjuntos de imágenes de diversos tamaños. Una vez entrenada, se analiza la capacidad de la red para memorizar y recuperar correctamente las imágenes aprendidas, incluso cuando estas han sido alteradas mediante la introducción de ruido o la eliminación de partes. Además, se exploran las relaciones entre la capacidad de aprendizaje de la red y su tamaño, así como la influencia de la correlación entre los patrones utilizados en el proceso de entrenamiento.

1.1 Red de Hopfield: Entrenamiento y Evaluación

Una red de Hopfield es un tipo de red neuronal en donde todas las neuronas están interconectadas entre si pero no consigo mismas. Las conexiones entre las neuronas se representan mediante una matriz W de pesos sinápticos en donde sus elementos, denominados w_{ij}, representan la conexión entre la i-esíma neurona con la j-esíma

Como la red de Hopfield no permite la conexión de una neurona consigo misma, $w_{ij} = 0$ cuando $i = j$, y tiene simetría en sus conexiones, es decir, $w_{ij} = w_{ji}$. Por lo tanto, se concluye que W es una matriz simétrica con todos los elementos de su diagonal iguales a 0.

1.2 Regla de Hebb para el Entrenamiento

El entrenamiento de la red se realiza utilizando la regla de Hebb. La matriz de pesos sinápticos w_{ij} se ajusta de acuerdo a los patrones de entrenamiento ξ^μ, siguiendo la siguiente ecuación:

$$\Delta w_{ij} = \sum_{\mu=1}^{P} \xi_i^\mu \xi_j^\mu \quad \text{para} \quad i \neq j \tag{1}$$

Donde:

- P es el número de patrones de entrenamiento,
- ξ_i^μ es el valor del patrón μ en la neurona i.

1.3 Evaluación de la Red

Después de ser entrenada, la red de Hopfield se puede evaluar de manera sincrónica o asincrónica.

1.3.1 Evaluación Sincrónica

En la evaluación sincrónica, todas las neuronas se actualizan simultáneamente en función de las entradas actuales. La actualización de cada neurona se realiza usando la siguiente regla:

$$X_i(t+1) = \text{sgn}\left(h_i(t+1)\right) = \text{sgn}\left(\sum_j w_{ij} X_j(t)\right) = \text{sgn}\left(\mathbf{W} \cdot \mathbf{X}\right) \tag{2}$$

Donde:

- $X_i(t+1)$ es el nuevo estado de i-esíma neurona en el tiempo t+1,
- $X_j(t)$ es el estado de la j-esíma neurona en el tiempo t,
- $\mathbf{W}$ es la matriz de pesos sinápticos,
- $\text{sgn}(x)$ es la función signo, definida como:

$$\text{sgn}(x) = \begin{cases} +1 & \text{si } x \geq 0 \\ -1 & \text{si } x < 0 \end{cases}$$

1.3.2 Evaluación Asincrónica

En la evaluación asincrónica, la regla de actualización es la misma que de forma sincrónica solo que las neuronas se actualizan una a la vez, en un orden aleatorio.

Esta actualización se repite hasta que el estado de la red deja de cambiar, lo que significa que ha alcanzado un estado de equilibrio o un mínimo en la función de energía de la red de Hopfield.

1.4 Implementación de la Red de Hopfield

La red neuronal se implemento en el lenguaje de programación Python de la siguiente forma:

```
class HopfieldNetwork :
    def __init__(self, num_neurons):
        self.num_neurons = num_neurons
        self.weights = np.zeros((num_neurons, num_neurons))


    def train (self, patterns):
        self.weights += patterns @ patterns.T
        np.fill_diagonal(self.weights, 0)


    def evaluate_asynchronous (self, inicialInput):
        state = inicialInput
        previousState= np.zeros((self.num_neurons, 1))
        cicles = 0
        while not (np.array_equal(state, previousState)):
            previousState = state
            indices = np.random.permutation(self.num_neurons)
            for i in indices :
                sum = np.dot(self.weights[i,:], state)
                state[i] = np.where(sum > 0,1,-1)
            cicles += 1
        return state,cicles


    def evaluate_synchronous(self, pattern):
        sum_values = self.weights @ pattern
        return np.where(sum_values > 0,1,-1)
```

Es importante destacar que existen dos métodos para evaluar la red: de forma sincrónica y de forma asincrónica. La evaluación sincrónica se empleará exclusivamente para determinar la capacidad de aprendizaje en función del tamaño de la red o la correlación entre los patrones aprendidos y consta de un unico ciclo de actualizacion donde todas las neuronas se actualizan a la vez. Por otro lado, la evaluación asincrónica, que actualiza las neuronas de manera individual y en un orden aleatorio hasta que el estado global de la red se mantenga estable entre dos ciclos consecutivos de actualización, será utilizada para los demás análisis.

2 Red Neuronal de Hopfield para set de imágenes 50x50

2.1 Entrenamiento

Se creó una instancia de la clase **NetworkHopfield** denominada **Red1** con el propósito de entrenarla y evaluarla exclusivamente con imágenes de tamaño 50×50 píxeles. Como cada imagen es un patrón y cada píxel es un elemento de ese patrón, la **Red1** estará compuesta por 2500 neuronas. El set de imágenes utilizado para el entrenamiento de la **Red1** es el siguiente:

Figure 1: Set 1 de imagenes: 50x50 pixeles

Una vez que se entrenó la red con las imágenes de la Fig. 1, se evaluó utilizando las mismas imágenes para comprobar si las había aprendido correctamente. Como se observa en la Fig. 2, la red logra reproducirlas de manera precisa.

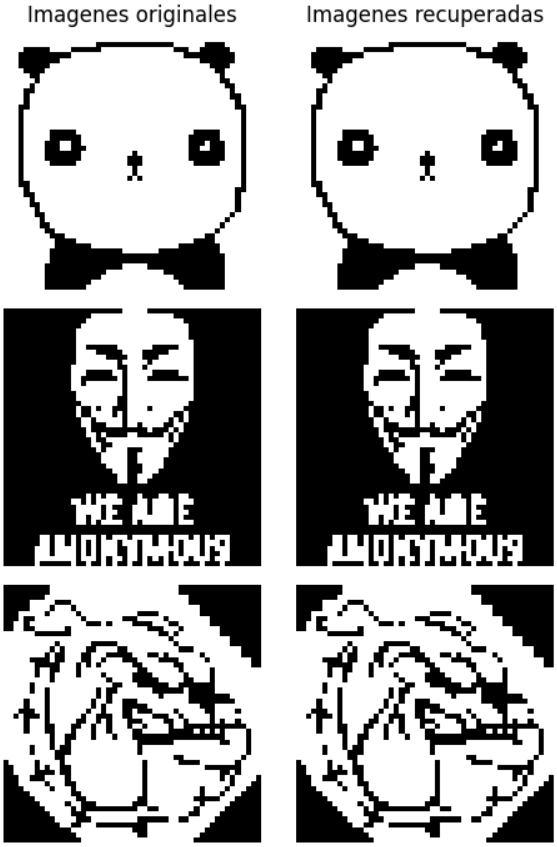
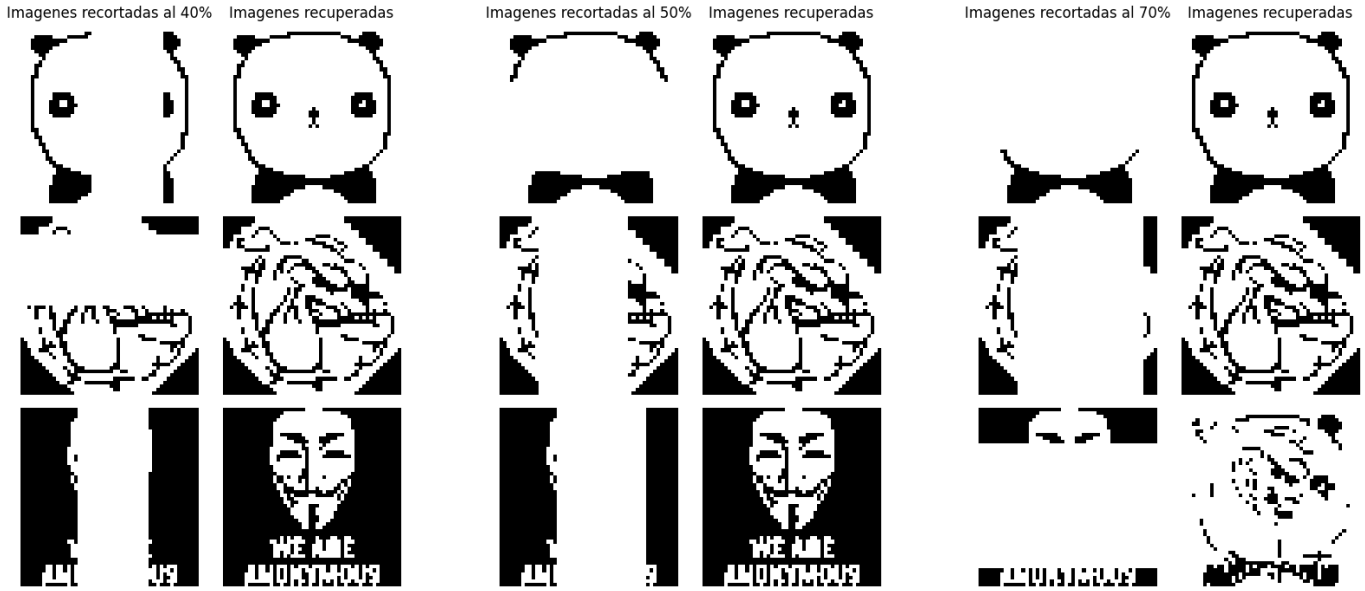


Figure 2: **red1** evaluada en el set 1 de imagenes

2.2 Evaluación de la Red con imágenes recortadas o con ruido

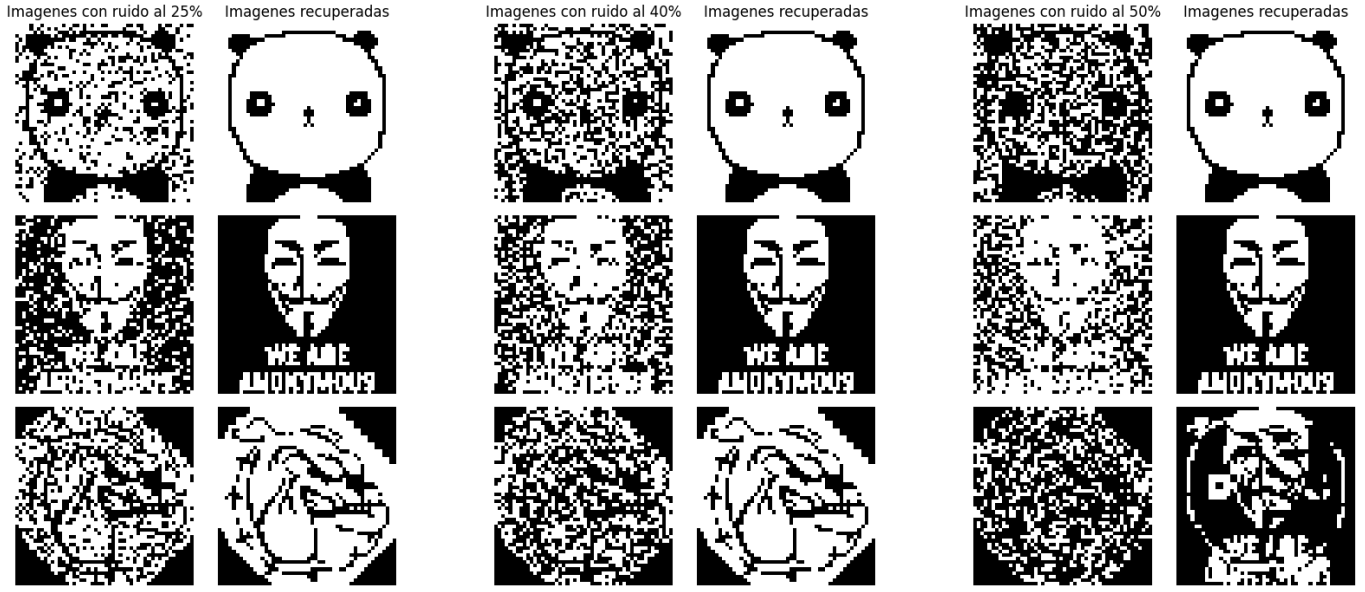
Para comprobar la capacidad de la red de recuperar las imágenes aprendidas, se evaluó la **Red1** con las imágenes de entrenamiento pero con un determinado porcentaje de recorte o ruido.

En las Fig. 10 y 11 se observa los resultados obtenidos al evaluar la red con las imágenes del set de entrenamiento pero con ruido o recortes.



(a) Imágenes recortadas al 40% (b) Imágenes recortadas al 50% (c) Imágenes recortadas al 70%

Figure 3: Resultados obtenidos de evaluar la **Red1** con imágenes recortadas



(a) Imágenes con ruido al 25% (b) Imágenes con ruido al 40% (c) Imágenes con ruido al 50%

Figure 4: Resultados obtenidos de evaluar la **Red1** con imágenes ruidosas

Se observan que en el caso del recorte de imágenes, la **Red1** puede recuperar las imágenes originales cuando se evalúan las mismas recortadas hasta un 70% o con un ruido aleatorio de hasta 50%.

Tanto en el caso del ruido como en el de los recortes, se recuerda que, durante el entrenamiento, la red de Hopfield no solo aprendió los patrones originales, sino también estados no deseados, o también espurios los cuales se entrara mas en detalle mas adelante, de esos mismos patrones. Si el porcentaje de ruido o el nivel de recorte es tan elevado que la imagen a evaluar se desvía considerablemente de la imagen original, la red puede tener dificultades para realizar una recuperación correcta y, en su lugar, recuperar uno de estos estados no deseados o también mostrar algún estado incorrecto. Esto es especialmente problemático si las alteraciones afectan a los píxeles que tienen los mayores pesos sinápticos asociados.

Los pesos sinápticos indican la importancia de las conexiones entre las neuronas, y si se modifica la información en los píxeles más críticos, la red puede converger hacia uno de estos estados no deseados, o mejor dicho espurios, en lugar de recuperar el patrón deseado. También, la modificación de la información puede resultar en confusiones donde la red interpreta erróneamente la imagen original, llevándola a recordar patrones distorsionados o no aprendidos. Así, tanto el ruido como el recorte en la imagen pueden dificultar el proceso de recuperación, haciendo que la red se desvíe de su objetivo de reconstruir la imagen original con precisión.

2.3 Estados Espurios

Luego, comprobamos la existencia de los estados espurios aprendidos por la **Red1**. La aparición de estos estados espurios es porque en una red de Hopfield, la dinámica de la red está controlada por una función de energía que la red trata de minimizar mientras va evolucionando. Los patrones que la red aprende funcionan como puntos en los que la energía es mínima y la red tiende a estabilizarse en ellos. Pero además de estos puntos que queremos, también pueden aparecer otros mínimos locales que no corresponden a los patrones almacenados, y esos son los estados espurios. Estos son combinaciones impares de las imágenes del set de aprendizaje de la red. Se componen de la suma o resta impar de los patrones del set de entrenamiento.

En la Fig. 5 y 6 observamos los posibles estados espurios que pudo haber aprendido la red, el primero corresponde al complemento del set de aprendisaje y el segundo a todas las combinaciones lineales del set

Figure 5: Complemento del set de aprendizaje de la **Red1**

Figure 6: Combinaciones lineales del set de aprendizaje de la **Red1**

Finalmente, evaluando la **Red1** en las imágenes de las Fig. 5 y 6, obtenemos según la Fig. 7 que realmente la Red aprendió todos estos estados espurios.

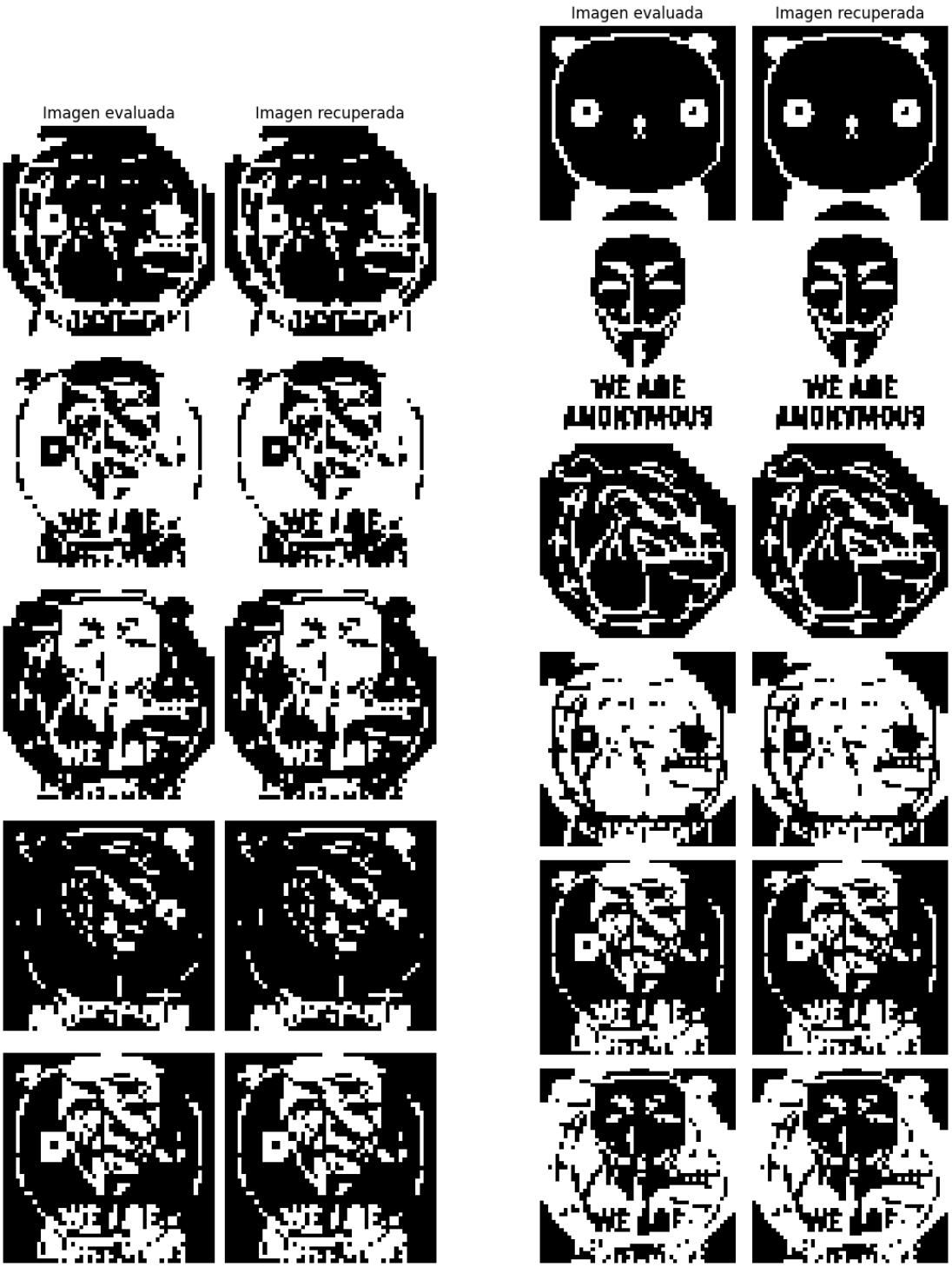


Figure 7: Estados espurios aprendidos por la **Red1**

3 Red Neuronal de Hopfield para set de imagenes 45x60

3.1 Entrenamiento

En esta sección se creara otra instancia de la clase **NetworkHopfield** denominada **Red2** para entrenarla y evaluarla con imágenes de 45x60 píxeles. Dado estas dimensiones de las imágenes para el set de entrenamiento, la **red2** contiene 2700 neuronas. El set de entrenamiento utilizado es el siguiente:

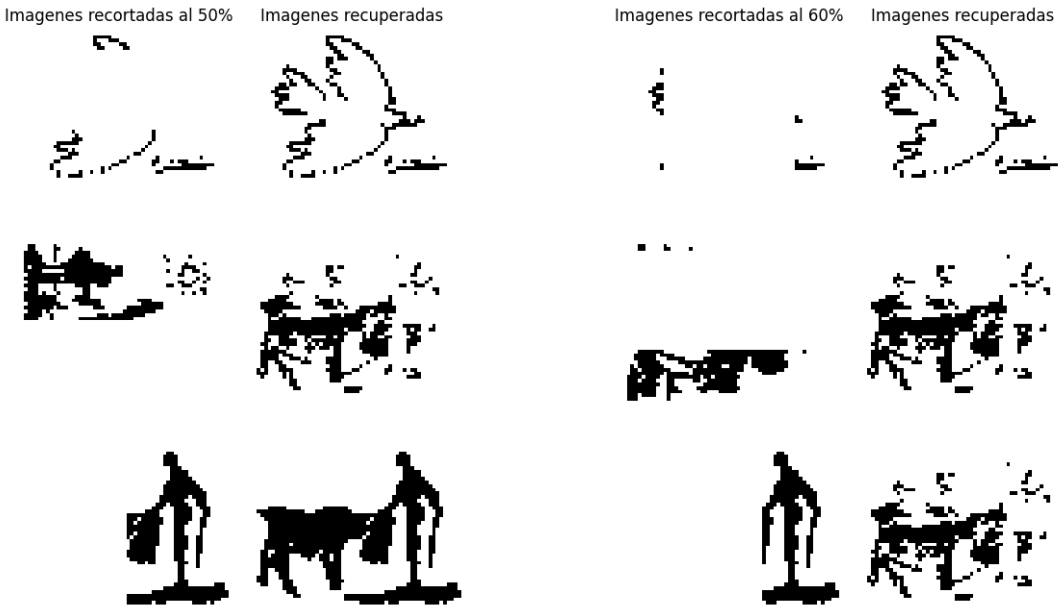
Figure 8: Set 2 de imagenes: 45x60 pixeles

Una vez entrenada la **red2** con las imágenes de la Fig. 8, análogamente a la **Red1**, se la evaluó con las mismas imágenes del set de entrenamiento para saber si las aprendió correctamente. Otra vez, vemos en la Fig. 9 como logra aprender el set perfectamente.

Figure 9: **red2** evaluada en el set 2 de imagenes

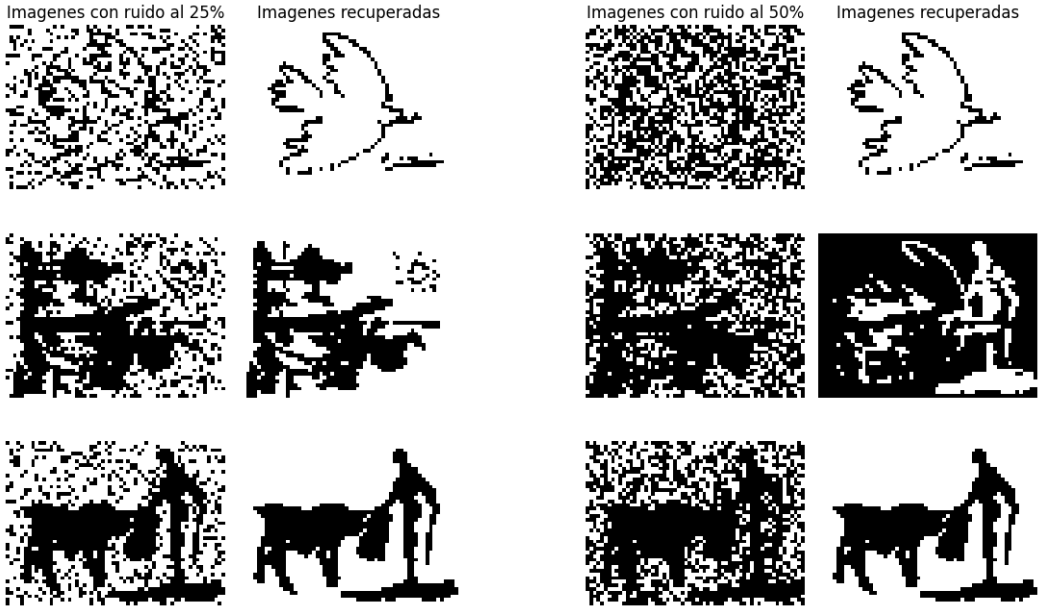
3.2 Evaluación de la Red con imágenes recortadas o con ruido

Al igual que con la **Red1**, se evaluara la **Red2** con imagenes de su set de entrenamiento recortadas o con ruido.



(a) Imágenes recortadas al 40% (b) Imágenes recortadas al 50%

Figure 10: Resultados obtenidos de evaluar la **Red2** con imágenes recortadas



(a) Imágenes con ruido al 25% (b) Imágenes con ruido al 50%

Figure 11: Resultados obtenidos de evaluar la **Red2** con imágenes ruidosas

Se observa que al igual que para la **Red1**, la **Red2** para un determinado nivel de ruido o recorte, no puede recuperar ciertos patrones y en su lugar obtenemos otro patrón aprendido o algún estado espurio.

3.3 Estados Espurios

Finalmente, también se comprueba la existencia de estados espurios aprendidos por la **Red2**.

En las Fig. 12 y 13, se tienen todos las posibles combinaciones impares del set de imágenes 2.

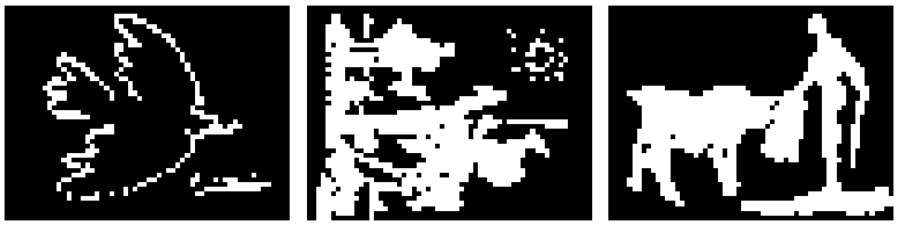


Figure 12: Complemento del set de aprendizaje de la **Red2**

Figure 13: Combinaciones lineales del set de aprendizaje de la **Red2**

En la Fig. 14 vemos como aprende perfectamente todas estas combinaciones impares de las imágenes del set 2.

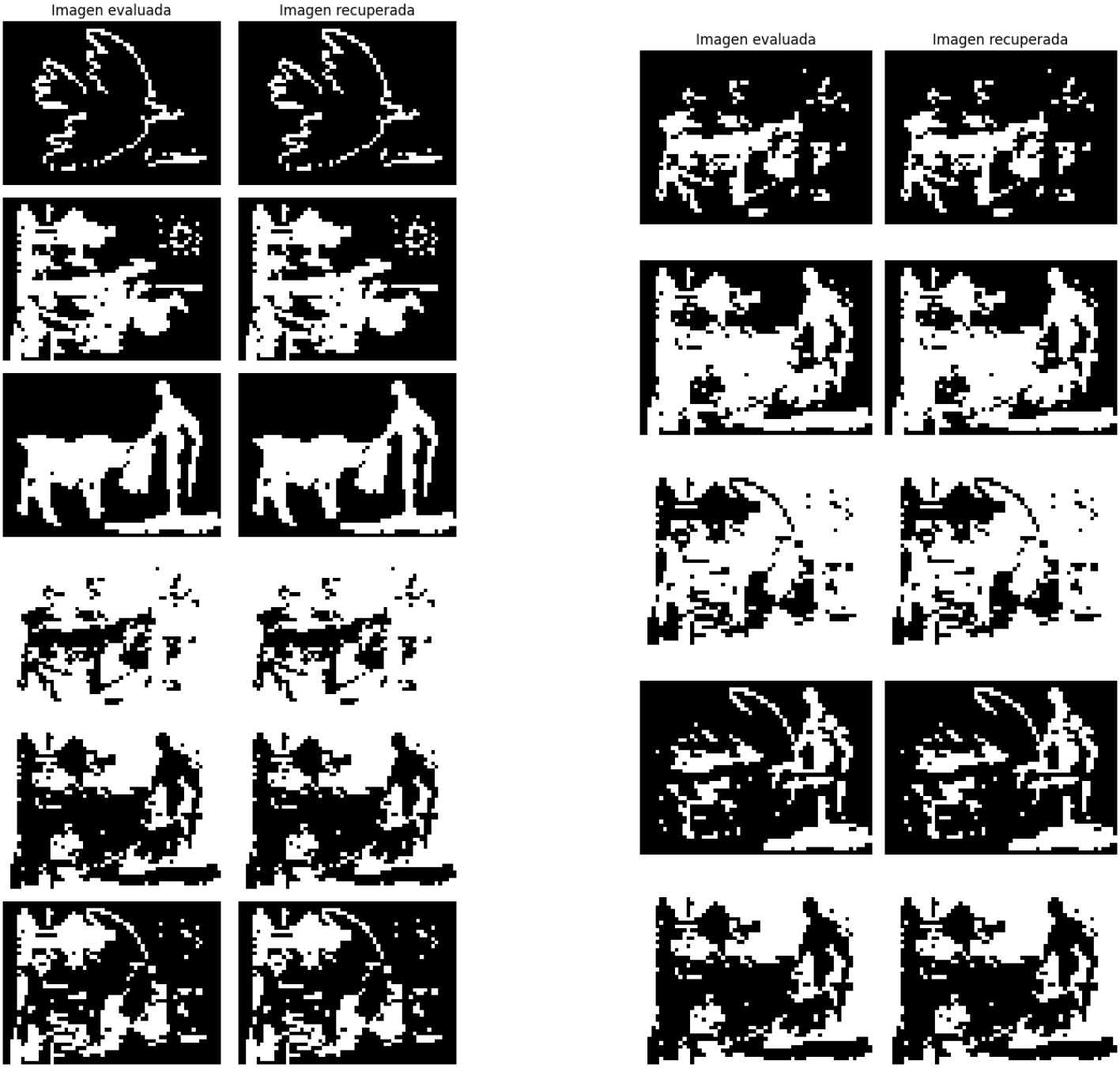


Figure 14: Estados espurios aprendidos por la **Red2**

4 Red Neuronal de Hopfield para set de imágenes de varios tamaños

A modo de probar la capacidad de una red de Hopfield para aprender patrones redimensionados, se creó la instancia **Red3** de la clase **NetworkHopfield**.

Para poder enseñarle a la red un conjunto de imágenes de distintos tamaños, todas las imágenes fueron redimensionadas a una misma dimensión. Se eligió que las dimensiones de las imágenes escaladas correspondieran a la mayor dimensión de ambos conjuntos, siendo esta de 50x60 píxeles.

El método de escalado utilizado fue INTER_NEAREST de la biblioteca de Python OpenCV. Este es un método de interpolación que se emplea para redimensionar imágenes, siendo el más sencillo y rápido de los disponibles en OpenCV, ya que no realiza cálculos complejos para estimar los valores de los nuevos píxeles. En su lugar, simplemente asigna a cada nuevo píxel el valor del píxel más cercano en la imagen original. Este método es ideal cuando se necesita escalar una imagen sin suavizarla, lo cual es útil para imágenes con bordes definidos o imágenes binarias (como las que se utilizan en esta red de Hopfield). Sin embargo, este tipo de interpolación puede generar artefactos visuales en imágenes con detalles finos o gradientes suaves.

En la Fig. 15 se muestra el conjunto final de imágenes utilizado para entrenar la **Red3**.

Figure 15: Conjunto de entrenamiento de la **Red3**.

Finalmente, evaluamos la **Red3** con el conjunto de entrenamiento, obteniendo los resultados mostrados en la Fig. 16.

Se observa que la red no logró aprender 3 de los 6 patrones. Esto puede deberse a diversas razones, entre ellas la capacidad máxima de aprendizaje de la red o el hecho de que la interpolación utilizada para redimensionar las imágenes puede afectar la correlación entre los patrones. Ambos factores influyen en la capacidad máxima de aprendizaje de la red, como se demostrará más adelante. Además, al añadir más imágenes, también se incrementa la cantidad de estados espurios que la red puede aprender, lo cual puede provocar que, al evaluarla con una imagen alterada, la red tienda a converger hacia uno de estos.

Figure 16: **Red3** evaluada con el conjunto de imágenes escaladas.

5 Capacidad de la Red Hopfield

En esta sección se analizará la capacidad máxima de una red de Hopfield con N neuronas. Para determinar esta capacidad, es necesario definir un error de aprendizaje. El error de aprendizaje se calcula durante un único ciclo de evaluación sincrónica y se mide para cada patrón que se ha enseñado a la red. El proceso consiste en evaluar la red con todos los patrones aprendidos y comparar los patrones obtenidos con los originales que se usaron durante el entrenamiento. El error se define como la cantidad de diferencias entre los patrones obtenidos y los enseñados, dividido por el producto entre el número total de patrones y la cantidad de neuronas.

El error de aprendizaje, denotado como $E_{\text{aprendizaje}}$, se calcula mediante la siguiente expresión:

$$E_{\text{aprendizaje}} = \frac{1}{N \cdot P} \sum_{i=1}^{P} \sum_{j=1}^{N} \left| \xi_{ij} - \hat{\xi}_{ij} \right|$$

Donde:

- P es el número total de patrones.
- N es la cantidad de neuronas en la red.
- ξ_{ij} representa el valor del j-ésimo elemento del patrón i enseñado.
- $\hat{\xi}_{ij}$ representa el valor del j-ésimo elemento del patrón i obtenido tras la evaluación.
- La suma $\sum$ mide las diferencias entre los valores originales y obtenidos.

La capacidad máxima de la red se define para un error máximo permitido, P_{error}, y corresponde al número de patrones que la red puede aprender, P_{max}, antes de alcanzar dicho error. Finalmente, la capacidad se expresa como la relación entre P_{max} y la cantidad de neuronas N:

$$\text{capacidad} = \frac{P_{\text{max}}}{N}$$

Se busca obtener experimentalmente la siguiente tabla que relaciona la capacidad de aprendizaje de una Red Hopflied, $\frac{P_{max}}{N}$ para un determinado error maximo, P_{error}.

P_{error}	$\frac{P_{\text{max}}}{N}$
0,001	0,105
0,0036	0,138
0,01	0,185
0,05	0,37
0,1	0,61

Table 1: Relación entre P_{error} y $\frac{P_{\text{max}}}{N}$

Para esto, se implementa la siguiente función en el lenguaje Python que a partir de patrones formados de secuencias de -1 y 1 entrena y evalúa la red en estos patrones y obtiene la capacidad de la red para un determinado P_{error}

```python
def capacity_by_error(P_error, N):
    hopfield_net = HopfieldNetwork(N)
    patterns_accumulated = np.empty((N, 0))
    error_neurons = 0

    while error_neurons < P_error:
        new_pattern = np.random.choice([-1, 1], size=(N,)).reshape(N, 1)
        hopfield_net.train(new_pattern)
        patterns_accumulated = np.hstack((patterns_accumulated, new_pattern))
        evaluated_patterns = hopfield_net.evaluate_synchronous(patterns_accumulated)
        error_neurons = np.count_nonzero(evaluated_patterns- patterns_accumulated) / (
            patterns_accumulated.shape[1]*N)

    capacity = (patterns_accumulated.shape[1]-1)/N
    return capacity
```

Se estima la capacidad para $P_{error} \in \{0.001, 0.0036, 0.01, 0.05, 0.1\}$ y para los casos de N=500 y N= 1500, obteniéndose las Tablas 1 y 2.

P_{error}	$\frac{P_{\text{max}}}{N}$
0,001	0,108
0,0036	0,134
0,01	0,188
0,05	0,374
0,1	0,604

Tabla 1: estimación $\frac{P_{\text{max}}}{N}$ para N=500

P_{error}	$\frac{P_{\text{max}}}{N}$
0,001	0,106
0,004	0,14
0,015	0,1853
0,055	0,37
0,12	0,6093

Tabla 2: estimación $\frac{P_{\text{max}}}{N}$ para N=1500

Dado que para una determinada cota del error P_{error} la capacidad $\frac{P_{max}}{N}$ tiende a un valor, es de esperar que si para cada P_{error} se grafica la curva P_{max} en función de N, se observara una tendencia lineal.

En la Fig. 17 se observa que para $P_{error} \in \{0.001, 0.0036, 0.01, 0.05, 0.1\}$, la curva P_{max} en funcion de N tiene efectivamente una tendencia lineal.

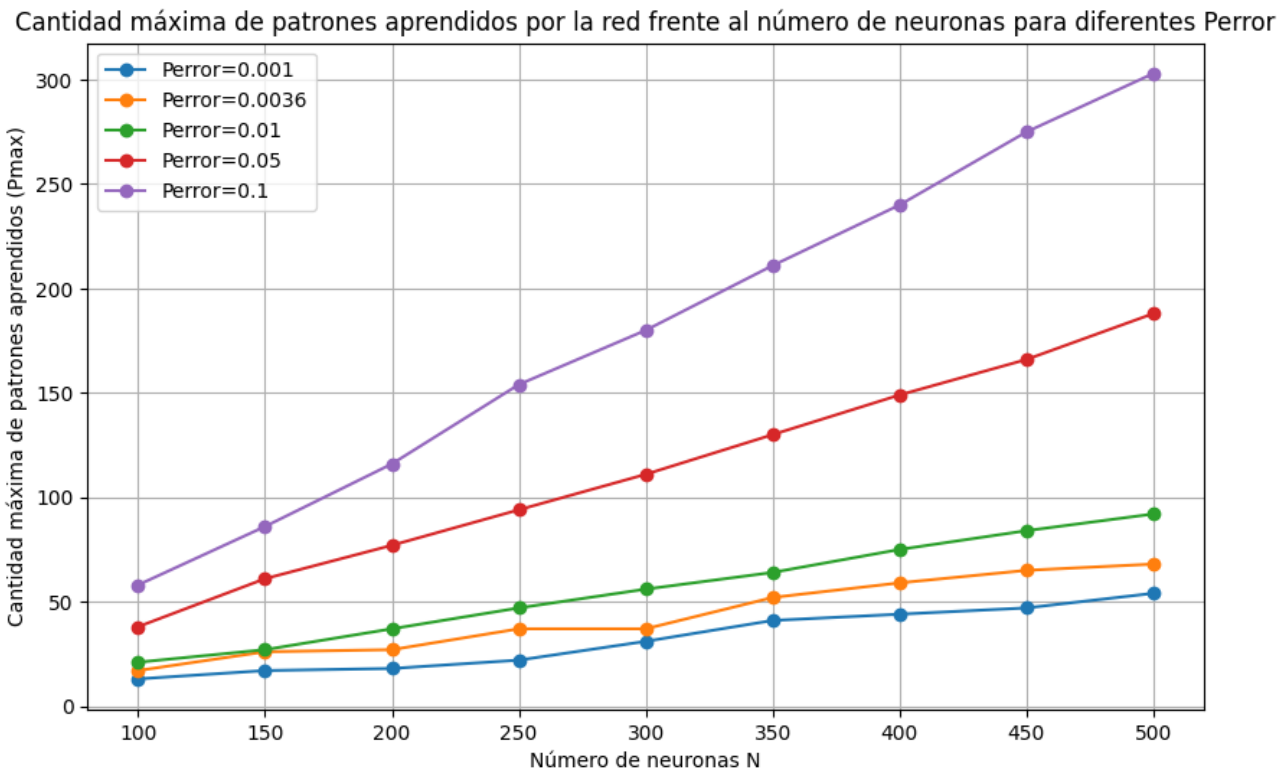


Figure 17: P_{max} en función de la cantidad de neuronas N

Finalmente, se busca analizar como varia la capacidad de aprendizaje de la red de Hopfield en función de la correlación de los patrones utilizados en el entrenamiento.

Para generar patrones correlacionados con los cuales se va a estimar la capacidad máxima de la red para ese nivel de correlación se utiliza la siguiente función implementanda en Python

```
def generate_correlated_patterns_bernoulli(base_pattern, correlation, num_patterns):
    N = len(base_pattern)
    correlated_patterns = np.empty((N, num_patterns))

    for i in range(num_patterns):
        for j in range(N):
            prob = (1 + correlation) / 2 if base_pattern[j] == 1 else (1 - correlation) / 2
            correlated_patterns[j, i] = 1 if np.random.rand() < prob else -1

    return correlated_patterns
```

La clave para generar patrones con un nivel determinado de correlación radica en la probabilidad con la cual un elemento j de un nuevo patrón es igual al correspondiente elemento j de un patrón base. Esta probabilidad se define de la siguiente manera:

$$P\left(x_j^{\text{nuevo}} = 1 \mid x_j^{\text{base}} = 1\right) = \frac{1+c}{2}$$

Una vez calculada esta probabilidad para un valor de correlación c dado, generamos secuencias aleatorias de 0s y 1s. Posteriormente, comparamos cada uno de estos valores con la probabilidad obtenida: si el valor generado es menor que la probabilidad, asignamos al elemento el valor 1; en caso contrario, asignamos el valor -1.

De esta manera, es posible crear nuevos patrones de 1s y -1s que mantienen un cierto nivel de correlación con el patrón base, modulando la similitud entre los patrones generados en función de c.

A partir de esto, se grafica en la Fig. 18 la curva de capacidad de aprendizaje de la red en función de la correlación de los patrones de entrenamiento para un determinado P_{error} y con $N = 500$ neuronas

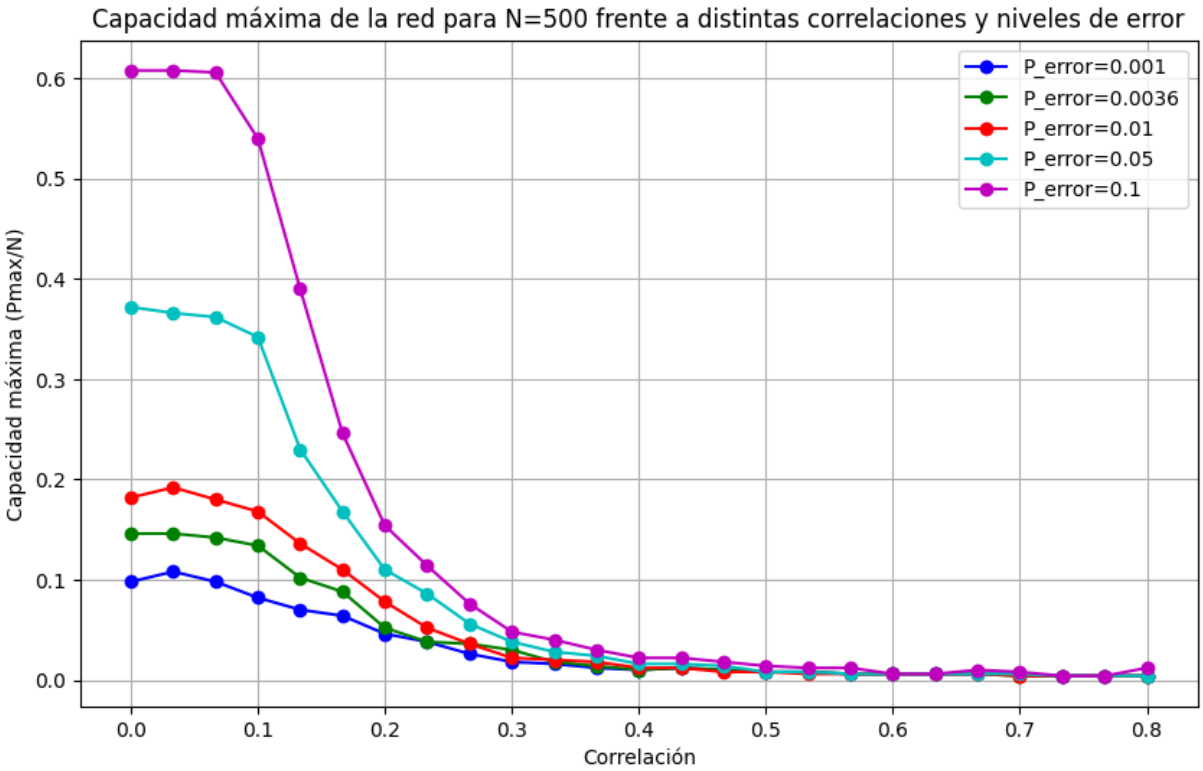


Figure 18: $\frac{P_{max}}{N}$ en función de la correlación entre patrones

Se observa que, a medida que la correlación entre los patrones aumenta, la capacidad de la red disminuye. Esto puede interpretarse como que, al entrenar la red con patrones más semejantes entre sí, la red tiende a confundirse más fácilmente, lo que provoca un mayor margen de error. Como consecuencia, la red es capaz de aprender correctamente (según una cota de error determinada) un número menor de patrones.

Es importante destacar que un caso particular ocurre cuando la correlación entre los patrones es igual a 1. En esta situación, como se muestra en la Fig. 18, la capacidad de la red tendería a infinito. Esto se debe a que, si $c = 1$, los patrones entrenados y evaluados son idénticos, lo que significa que el error no aumenta a medida que se agregan más patrones. Por lo tanto, el error nunca alcanza el umbral máximo permitido P_{error}, lo que permite que la red siga aprendiendo indefinidamente sin alcanzar el límite de error.